

■ Spring Security Roadmap

Basic → Advanced → Industry Level

Goal: Build a strong understanding of Spring Security from fundamentals through JWT, OAuth2/OIDC, enterprise identity, microservices security, and production architecture.

1. Complete Roadmap

Stage	Topics	What You Should Learn
1. Prerequisites	Spring Boot basics; HTTP fundamentals; Authentication concepts; Password security	REST API, Controller, Service, Repository, DTO, Exception Handling; HTTP methods, headers, status codes, cookies; Authentication vs Authorization; hashing, salting, BCrypt
2. Spring Security Fundamentals	Security architecture; Dependencies; Default security; SecurityFilterChain; Authorization	Security Filter Chain, filters, SecurityContext; spring-boot-starter-security; default login; modern Java configuration; authenticated(), permitAll(), hasRole(), hasAuthority()
3. Users & Passwords	UserDetails; UserDetailsService; PasswordEncoder; AuthenticationManager; Database authentication; Roles & authorities	Load users from database; BCrypt password hashing; authentication processing; MySQL/PostgreSQL + JPA; ROLE_USER, ROLE_ADMIN and permissions
4. REST API Security	Stateless security; Session vs Stateless; HTTP Basic; CSRF; CORS; Exception handling	REST + Spring Security; when to use sessions; Basic authentication; CSRF purpose; frontend/backend CORS; 401 vs 403
5. JWT Authentication	JWT fundamentals; Login API; Token generation; Token validation; JWT filter; Refresh tokens; Logout	Header/payload/signature; /auth/login; JWT creation and validation; OncePerRequestFilter; access + refresh tokens; token invalidation strategies
6. Role-Based Security	RBAC; URL authorization; Method security; @Secured; Permissions	Role-based access; requestMatchers(); @PreAuthorize; @Secured; fine-grained permissions
7. OAuth 2.0	OAuth2 concepts; Authorization Code flow; Client registration; OAuth2 Login; Access tokens	OAuth2 roles; external provider login; Google/GitHub-style providers; token handling
8. OpenID Connect	OIDC; ID Token; UserInfo; Claims	Authentication on OAuth2; identity information; user profile; identity/authorization claims
9. Resource Server	OAuth2 Resource Server; JWT Resource Server; Opaque tokens; Scopes	Secure APIs with bearer tokens; external JWT validation; introspection; read/write scopes
10. Authorization Server	Authorization Server concepts; OAuth2 flows; Tokens; Clients; Consent	Identity provider architecture; Authorization Code and Client Credentials; access/refresh/ID tokens; application clients; user consent
11. Advanced Authorization	RBAC; ABAC; Permission-based security; Method security; Custom authorization; Multi-tenant security	Fine-grained policies; AuthorizationManager; tenant-aware authorization
12. Production Security	Password policies; Brute-force protection; Rate limiting; Security headers; HTTPS; Secrets management; Logging; Auditing	Secure password rules; login attempt control; abuse prevention; browser security; TLS; environment/Vault secrets; security events; audit trails
13. Enterprise Security	SSO; Identity Provider; LDAP; Active Directory; SAML; OAuth2/OIDC	Single Sign-On; Keycloak/Auth0/Okta/Microsoft Entra ID; enterprise directory authentication; modern and enterprise federation
14. Microservices Security	API Gateway; Service-to-service security; JWT propagation; OAuth2 Resource Servers; mTLS; Service identity	Centralized security; internal authentication; identity propagation; workload authentication
15. Advanced Production Architecture	Zero Trust; Token rotation; Key rotation; JWKS; Distributed sessions; Centralized identity; Security monitoring	Never trust by default; token lifecycle; signing-key rotation; public-key distribution; Redis sessions; IdP architecture; SIEM/security events
16. Testing	Security testing; @WithMockUser; MockMvc; JWT testing; OAuth2 testing	Test secured endpoints, authentication, JWT flows, and OAuth2 flows
17. Industry Projects	JWT system; RBAC app; OAuth2 app; Microservices security; Enterprise SSO; Multi-tenant app	Build progressively larger projects using the concepts above

2. Recommended Learning Path

Level	Focus	Core Topics
■ Level 1	Spring Security Fundamentals	Authentication; Authorization; Security Filter Chain; SecurityContext; UserDetails; UserDetailsService; PasswordEncoder; AuthenticationManager; SecurityFilterChain
■ Level 2	Database Authentication	User entity; UserRepository; CustomUserDetailsService; UserDetails; PasswordEncoder; AuthenticationManager; MySQL/PostgreSQL
■ Level 3	JWT Security	JWT structure; generation; validation; expiration; JWT filter; Bearer token; access token; refresh token; token rotation; logout
■ Level 4	Authorization	URL-level authorization; roles; authorities; @PreAuthorize; @Secured; permissions; custom authorization; AuthorizationManager
■ Level 5	OAuth2 + OpenID Connect	OAuth2; OAuth2 Client; OAuth2 Login; Resource Server; OIDC; access token; refresh token; ID token; scopes; claims; Authorization Code; Client Credentials
■ Level 6	Enterprise / Production Security	Keycloak; Auth0; Okta; Microsoft Entra ID; SSO; LDAP; Active Directory; SAML; API Gateway; service-to-service authentication; JWT propagation; OAuth2 Resource Server; mTLS; Zero Trust

3. Level 1 — Spring Security Fundamentals

Core flow:

Authentication → Authorization → Security Filter Chain → SecurityContext → UserDetails → UserDetailsService → PasswordEncoder → AuthenticationManager → SecurityFilterChain

Important classes/interfaces: SecurityFilterChain, UserDetails, UserDetailsService, PasswordEncoder, Authentication, AuthenticationManager, SecurityContext, SecurityContextHolder

First project: Basic Spring Boot Login + Role-Based Authorization.

Example endpoints: POST /register, POST /login, GET /users, GET /admin, GET /profile

4. Level 2 — Database Authentication

Architecture:

Client → Spring Security → UserDetailsService → UserRepository → MySQL

User entity: id, username, email, password, enabled, role

Flow: User → UserRepository → CustomUserDetailsService → UserDetails → PasswordEncoder → AuthenticationManager

5. Level 3 — JWT Security

JWT authentication flow:

LOGIN → username/password → AuthenticationManager → UserDetails → Generate JWT → Client → Authorization: Bearer <JWT> → JWT Authentication → SecurityContext → Controller

JWT structure: Header.Payload.Signature

Example payload:

```
{"sub":"john","roles":["USER"],"iat":1727000000,"exp":1727003600}
```

Learn: JWT generation, validation, expiration, filter, Bearer token, access token, refresh token, token rotation, logout.

6. Level 4 — Authorization

URL-level authorization example:

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**").permitAll()
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/user/**").hasRole("USER")
    .anyRequest().authenticated());
```

Method-level authorization:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { /* delete */ }
```

Progression: Roles → Authorities → Permissions → Custom authorization → AuthorizationManager

7. Level 5 — OAuth2 + OpenID Connect

OAuth 2.0: Authorization and delegated access to resources.

OpenID Connect: Authentication + identity on top of OAuth 2.0.

Learn: OAuth2, OAuth2 Client, OAuth2 Login, OAuth2 Resource Server, OIDC, Access Token, Refresh Token, ID Token, Scopes, Claims, Authorization Code Flow, Client Credentials Flow.

Typical flow: React Application → Login → Keycloak/Identity Provider → Access Token → Spring Boot API → Validate Token → Allow / Deny

8. Level 6 — Enterprise / Production Security

Identity Providers: Keycloak, Auth0, Okta, Microsoft Entra ID

Enterprise authentication: SSO, LDAP, Active Directory, SAML, OAuth2, OIDC

Microservices: API Gateway security, service-to-service authentication, JWT propagation, OAuth2 Resource Server, Client Credentials, mTLS, service identity, centralized authentication

9. Production-Level Target Architecture

React Frontend → API Gateway → User Service / Order Service / Payment Service → Identity Provider (Keycloak/OIDC)

Security controls: HTTPS, JWT, OAuth2, OIDC, RBAC, scopes, mTLS, rate limiting, CORS, CSRF, security headers, audit logging, secret management, Zero Trust.

10. Important Spring Security Classes to Know

Category	Classes / Interfaces
Core	SecurityFilterChain; SecurityContext; SecurityContextHolder; Authentication; Principal; GrantedAuthority
Authentication	AuthenticationManager; AuthenticationProvider; DaoAuthenticationProvider; UserDetails; UserDetailsService; PasswordEncoder
Authorization	AuthorizationManager; GrantedAuthority; @PreAuthorize; @Secured
JWT / OAuth2	Jwt; JwtDecoder; JwtEncoder; BearerTokenAuthenticationFilter; JwtAuthenticationToken
OAuth2	OAuth2User; OAuth2UserService; ClientRegistration; OAuth2AuthenticationToken

11. Projects to Build

Project	Concepts
1. Basic Login System	Authentication
2. Database Login System	JPA + Security
3. Role-Based API	USER / ADMIN
4. JWT Authentication	JWT
5. JWT + Refresh Token	Token lifecycle
6. RBAC Application	Roles + authorities
7. Permission-Based API	Fine-grained authorization
8. Google OAuth2 Login	OAuth2
9. Keycloak + Spring Boot	OIDC
10. Secure Microservices	Resource Server
11. API Gateway Security	Gateway + JWT
12. Enterprise SSO	OIDC / SAML / IdP
13. Multi-Tenant Security	Tenant authorization
14. Production Security System	Full security architecture

12. Industry-Level Final Skill Set

Authentication: Passwords, UserDetails, JWT, sessions

Authorization: RBAC, ABAC, permissions, scopes, policies

Identity: OAuth2, OIDC, SSO, Keycloak, enterprise IdPs

Microservices: API Gateway, JWT/OAuth2, service identity, mTLS

Production: Secrets, HTTPS, CORS, CSRF, rate limiting, security headers, auditing, logging, monitoring, Zero Trust

13. Exact Recommended Sequence

Spring Security Basics → Authentication → Database Authentication → Roles / Authorities → JWT → Refresh Tokens → Method Security → CORS / CSRF → OAuth2 → OIDC → Resource Server → Keycloak → API Gateway → Microservices Security → Enterprise SSO → Production Security → Security Testing

Final goal: Move from understanding individual Spring Security annotations to designing and implementing secure, production-ready Spring Boot applications and microservices.